

core, you wouldn't even notice that there's some encoding going on in the background!

The Software Aspect

There's a difference between multi-tasking faster, and a single program running faster. You could expect a performance boost if, like we said, you're running two very different programs. But the story is different if you're running just one program. In this case, you may not even notice the difference between a regular processor and a dual-core.

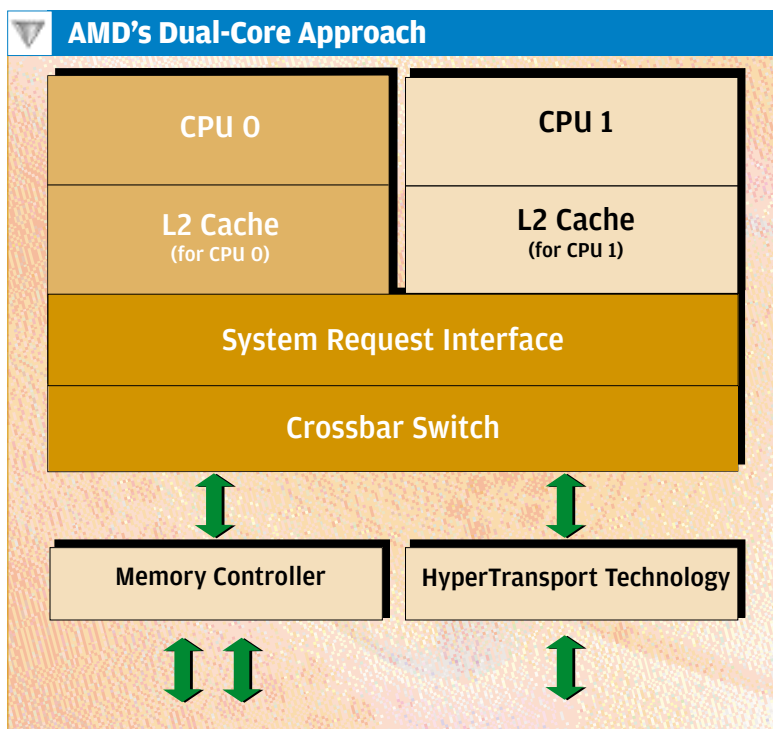
So overall, what kind of an improvement can you expect with dual-core? The operating system and the software have to be aware that there are two cores out there doing the work, and make the threads accordingly. Remember that the two cores make sense only if there are entirely different threads to exploit—so the separate threads have to be created somehow. And it is one of the jobs of the OS and the software to do this. On top of that, they obviously cannot do it with 100 per cent efficiency (or segregation).

So can current OSes and software do this? Unfortunately, not all. Some OSes, such as Windows XP Home, simply cannot acknowledge that there are two processor cores. And most games and applications aren't written with this in mind. (Adobe Photoshop, amongst other programs, happens to be, so if you use it extensively, go right out and buy a dual-core!)

Meta Group vice president Steve Kleynhans says, "It's a lot more complex to write software that's multithreaded. It's also harder to check for errors. We could find ourselves with buggier software. Certainly software vendors have known this and been working on multithreaded applications for some time, but it is more complex."

That makes it sound like a pretty dismal situation. But keep in mind two things—first, that as dual-core becomes mainstream, operating systems and software will definitely evolve towards being multithreaded, complex though it may be. The market will demand it. And secondly, as we mentioned before, even if your applications aren't currently aware of two processors, and if one app maxes out one core, the other core will prevent your system from lagging.

But then again, another question raises its head. If dual-processor-aware applications aren't



Because AMD added a Northbridge-like provision to the Athlon64 die to support the memory controller and the HyperTransport link, the two cores can communicate with each other within the processor itself. The 'crossbar switch' handles requests to the memory controller and the HyperTransport link to the rest of the system.

so many, why are the chipmakers pushing for their release right now, in their desktop avatars? The answer probably lies in competition—and early adopters of the technology apart, neither AMD nor Intel can afford to lag behind too much, lest its marketshare be gone by the time it gets in!

AMD's And Intel's Approaches

AMD's and Intel's approaches to dual-core is very different. In AMD dual-core processors, the two cores communicate via a Northbridge-like provision (see box *The Northbridge*) within the chip itself. AMD had to add this provision to the Athlon 64 die so as to support the onboard memory controller and HyperTransport link (see box *HyperTransport*), and this now comes in handy—a second core can easily be accommo-

Instructions And Pipelines

A computer instruction is not as simple as "add A and B". There's more to it than that. For example, if you're talking about A and B, the instruction needs to be read first; then, the data needs to be brought from wherever it is—either from the CPU registers, the CPU cache, or from main memory. After that, the addition operation must be done, and the result placed somewhere so it can be accessed.

There are independent units in the processor that do each of these things. For example, one unit fetches the instruction; another checks where in the system the values are (assuming it's an 'add' operation we're talking about); another brings the values in; another does the actual calculation; and finally, one unit places the result in a suitable location.

What happens in pipelining is that sequential instructions are carried out simultaneously, based on the principle that

each of the units mentioned above can do different things in different instructions.

To illustrate, suppose "add A and B" is followed by "add C and D". What happens? First, the "add A and B" instruction is fetched by the instruction fetcher. Then, while the appropriate unit is figuring out where A and B are, the "add C and D" instruction is fetched by the instruction fetcher. After that, while A and B are being added by the adding unit, the 'location figuring' unit is figuring where C and D are.

The basic idea of pipelining is as simple as that—each unit of the CPU is always doing something, instead of each instruction finishing before the next one is brought in.

Modern advances in computer architecture consist largely in figuring out how to make the pipeline architecture on the chip work better.